

Weighted heads, or fast state under Kraft

The dyadic machine of Expo 1 prices space by grade and movement by the tree metric, but it treats all heads alike: any number of them, each moving at unit cost. Real machines do not extend that courtesy. A pointer that stays hot lives in registers and low cache levels; keep many of them hot and they compete for the same fast silicon. This exposition adds that competition to D as a **priced, structural** resource — no cache simulator, no eviction model: one inequality and one surcharge.

1 The model

Every head i carries a weight $w_i \in \mathbb{N}$. Two rules (cache-v0, after convo [55]–[59]):

1. **Budget.** An assignment is admissible iff $\sum_i 2^{-w_i} \leq 1$ — the Kraft inequality. Head i occupies the fraction 2^{-w_i} of fast state; the shares are disjoint (no overlap between heads, by design: overlap is what makes sharing models treacherous).
2. **Surcharge.** Every operation of head i costs $1 + w_i$ instead of 1: the weight is the head’s address depth in an implicit fast-state tree — a lighter-weighted (hotter) head is cheaper to touch.

The plain machine is the fiber $w \equiv 0$:

LEAN $\vdash$ Adic.Dyadic.weightedCost_zero

Weighting is a refinement

At weight zero everywhere, weighted cost is exactly the old cost — every Expo 1 theorem survives as the $w \equiv 0$ fiber, and weighted cost remains a homomorphism on words.

```
theorem weightedCost_zero {k : Nat} (word : ActionWord k) :
  weightedCost (fun x => 0) word = actionCost word
axioms: propect      hash: ec00b0653fab
```

The Kraft condition is not an analogy but the region algebra itself: admissible weights are exactly antichains of dyadic mount slots.

LEAN $\vdash$ Adic.Dyadic.kraft_iff_mounting

Kraft = an antichain of mount slots

A weight assignment satisfies Kraft **iff** the heads can be mounted injectively on pairwise prefix-incomparable paths with $|\text{path}_i| = w_i$ — prefix-free codes, dyadic disjointness, and fast-state budgeting are one condition. Both directions, with a constructive canonical mounting.

```
theorem kraft_iff_mounting {k : Nat} (weights : Weights k) :
  Kraft0k weights  $\leftrightarrow$ 
   $\exists$  paths,
    ( $\forall$  (head : Fin k), (paths head).length = weights head)  $\wedge$ 
    Function.Injective paths  $\wedge$ 
     $\forall$  {left right : Fin k},
      left  $\neq$  right  $\rightarrow$  PrefixIncomparable (paths left) (paths right)
axioms: Classical.choice, Quot.sound, propect      hash: 95f1dbc459c2
```

2 Streaming survives weighting

LEAN $\vdash$ Adic.Dyadic.weightedZipCost_closed

Weighted zip, exact

The zip schedule of Expo 1 under weights has an exact closed-form cost, still linear in 2^n — the surcharge enters as the multiplicative $(1 + w)$ and nothing worse. The convo’s own worked example ([59]) — three heads at uniform weight 2, Kraft mass $\frac{3}{4}$ — is instantiated and checked:

```
theorem weightedZipCost_closed (weights : Weights 3) (n : Nat) :
  weightedZipCost weights n =
    weightedZipLeafCost weights * 2 ^ n +
    weightedZipNodeCost weights * (2 ^ n - 1)
axioms: Quot.sound, propect      hash: 1a624131020f
```

LEAN $\vdash$ Adic.Dyadic.uniformThreeWeights_kraft

...and [59]’s example is admissible

$3 \cdot 2^{-2} \leq 1$, in the all-natural formulation, no real numbers anywhere.

```
theorem uniformThreeWeights_kraft : Kraft0k uniformThreeWeights
axioms: Quot.sound, propect      hash: beecf57bdf70
```

3 The entropy bound

Which weights should a program choose? Count the touches: head i touched c_i times out of n . Choosing weights against inverse empirical frequency is admissible and essentially optimal — the fast-state allocation problem is source coding.

LEAN $\vdash$ Adic.Dyadic.empiricalWeights_kraft

Frequency weights are admissible

The inverse-frequency choice $w_i = \lceil \log_2(n/c_i) \rceil$ satisfies Kraft — the classical prefix-code argument, in $\mathbb{N}$.

```
theorem empiricalWeights_kraft {k : Nat} (touches : List (Fin k))
  (hnonempty : 0 < touches.length)
  (hpositive :  $\forall$  (head : Fin k), 0 < touchCount touches head) :
  Kraft0k (empiricalWeights touches)
axioms: Quot.sound, propect      hash: d692acac2be1
```

LEAN $\vdash$ Adic.Dyadic.touchCost_empirical_entropy_bound

Achievability: cost $\leq$ entropy + slack

Under those weights the total touch cost is bounded by the empirical entropy of the touch sequence (count form $\sum_i c_i \log(n/c_i)$) plus explicit linear slack — the $H(p) + O(1)$ per touch of convo [57], with every ceiling accounted.

```
theorem touchCost_empirical_entropy_bound {k : Nat} (touches : List (Fin k))
  (hnonempty : 0 < touches.length)
  (hpositive :  $\forall$  (head : Fin k), 0 < touchCount touches head) :
  touchCost (empiricalWeights touches) touches  $\leq$ 
    empiricalFloorLogTotal touches + 2 * touches.length
axioms: Quot.sound, propect      hash: 457dac133f84
```

LEAN $\vdash$ Adic.Dyadic.touchCost_entropy_lower

Optimality: no weights beat entropy

Conversely, **every** Kraft-admissible assignment pays at least the empirical entropy (floor form, explicit slack): Shannon’s lower bound, carried by a discrete Gibbs/log-sum inequality proved by the counting route —

```
theorem touchCost_entropy_lower {k : Nat} (weights : Weights k)
  (touches : List (Fin k)) (hKraft : Kraft0k weights) :
  touches.length + empiricalFloorLogTotal touches  $\leq$ 
    touchCost weights touches + touches.length
axioms: Quot.sound, propect      hash: f52ccc2b3713
```

LEAN $\vdash$ Adic.Dyadic.discrete_gibbs_log_sum

The discrete log-sum inequality

$\sum_i c_i \lceil \log_2(n/c_i) \rceil \leq \sum_i c_i w_i + n$ for any Kraft-admissible w — all-natural, zero-count heads included.

```
theorem discrete_gibbs_log_sum {k : Nat} (total : Nat)
  (counts : Fin k  $\rightarrow$  Nat) (weights : Weights k) (htotal : finSum counts = total)
  (hKraft : Kraft0k weights) :
  floorLogTotal total counts  $\leq$  weightedCountTotal counts weights + total
axioms: Quot.sound, propect      hash: 02a7eba433b8
```

Together: fast state allocated like a prefix code, priced like a code length, optimal like a Shannon code. Biased search trees (Bent–Sleator–Tarjan) are the data-structure ancestor; dynamic reweighting (Huffman/FGK) is a known road, deferred.

4 What this model refuses

DESK-PROVED — not mechanized

Kraft pricing is not cliff pricing

k -way merge under Kraft weights costs $n \log n$ for **every** k — the multi-way tradeoff cancels exactly ([57]’s honest computation). The external-memory win ($\log$ base $\frac{M}{B}$) comes from the capacity **cliff**, which this model deliberately lacks: it never under-charges, and pessimism is the right failure mode for a verifier. Mechanizing this cancellation is phase 3; until then it is desk-level.

UNFORMALIZED

The write-head weight ([72])

Should zip’s write head, moving twice the data, get a larger share? Touch-frequency pricing says no (all three heads touch equally often); data-rate pricing says yes. The tension is exactly touch-weighted vs size-weighted Kraft, and its resolution belongs to the size-weighted refinement (v1), where a head’s state size enters the budget.